

从个人经验到组织资产：小米AI for SRE 的闭环实践与资产沉淀

赵文成

小米运维平台研发负责人



极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

🕒 6月 📍 上海

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：6月26-27日

- AI Infra 系统工程
- 多 Agent 协作与实践
- 多模态融合
- 模型训练与推理创新
- 数据平台与特征服务

🕒 8月 📍 深圳

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：8月21-22日

- Agentic AI
- 轻量化与高效推理
- 多模态应用
- AI + IoT 场景实践
- AI 工业化落地

🕒 10月 📍 上海

👥 1200人

QCon

全球软件开发大会

会议时间：10月22-24日

- AI Agent
- Vibe Coding
- 智能可观测
- 推理基建
- 模型攻防
- AI x 创造力

🕒 12月 📍 北京

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：12月18-19日

- 大模型架构创新
- 多模态 AI 产业融合
- 具身智能
- AI for Science
- 大模型安全

目录

01 演进历史与痛点

02 SRE 知识资产沉淀

03 AIOps 平台简介

04 落地案例展示

05 经验总结与展望

01

演进历史与痛点

我们走过的路-我们的感悟

小米业务特点和主要痛点

业务形态广

- 互联网业务
- IoT业务
- 汽车 & 金融
- 电商 & 新零售

业务规模大

- 日活用户：亿级体量
- 系统架构：微服务化

运维核心痛点

- 异常告警 多
- 琐碎 Oncall 杂
- 应急恢复 慢
- 知识分散新人上手难
- 快速迭代稳定压力大



AI 时代 SRE 变与不变



不变：SRE 的本质

质量：对系统确定性、可用性的极致追求

成本：对资源与效率的持续度量与优化

效率：将人力从重复、琐碎的工作中释放

变：工程化方法

手段：从脚本、工具到 Agent workflow

挑战：大模型的不确定性与 SRE 工作的确定性矛盾

知识：从让人能找到到让机器能找到

三阶段探索：工程能力的探索与升级

Function Call

实现连接，AI可以使用基础能力了

MCP

实现工具的兼容性和模型的迁移能力

Agent

尝试平台化与体系化





困局：打磨了“手脚”，忽视了“大脑”

核心

统一的困局

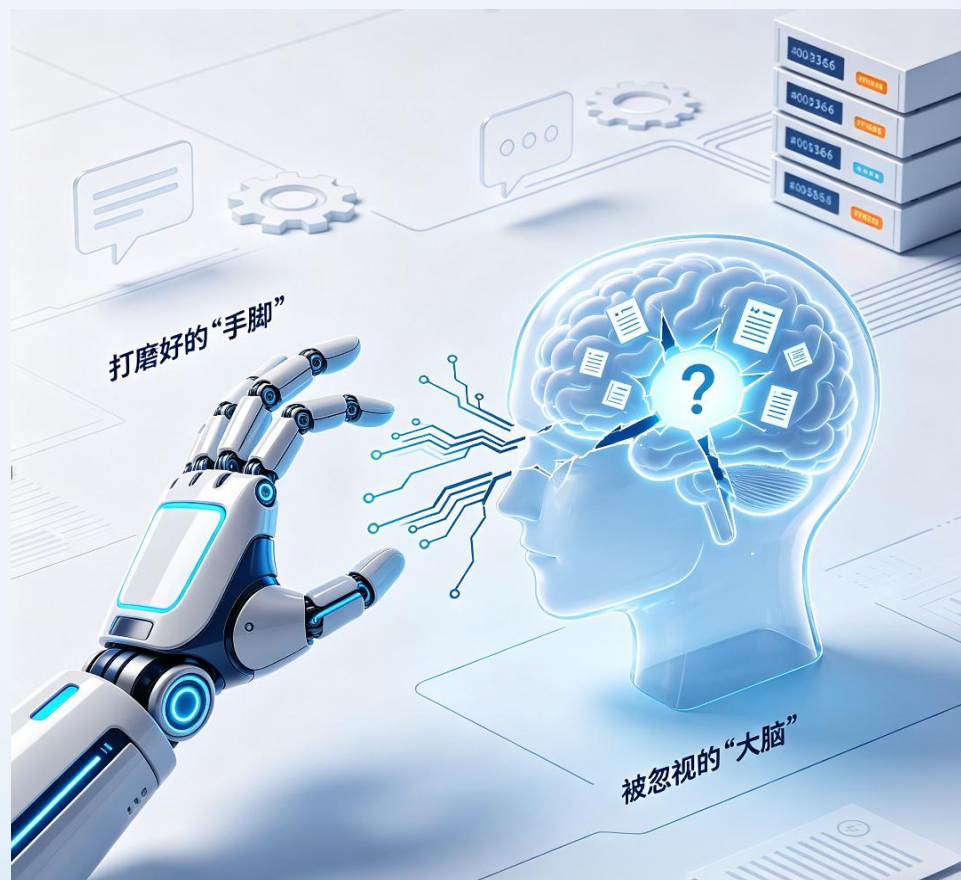
- 智能场景碎片化：AI 不知道在什么情况下以什么顺序调用
- 有工具无策略：有了标准的工具箱，什么时候使用什么工具的知识依然散落在 SRE 大脑中
- 有躯壳无灵魂：高度依赖提供的提示词和思维链

仅有“手脚”难落地

- 业务形态多，通用难度极大
- 流程制定成本高
- 微调复杂，结果难以保证
- 用户耐心有限，没有动力联调
- 平台工程与 SRE 一线工作不匹配

瓶颈不是模型是环境：

工程化的重点是融合 SRE 隐性知识、经验与判断逻辑



02 SRE 知识资产沉淀

为AI注入灵魂的工程实践

SRE资产体系的三个主体

工程化的目标是管理好这几个元素

技能 (Skills)

可被调用的原子化操作能力

建设思路：分层建设，分为基础skills + 业务skills

知识库 (Knowledge Base)

体系化的静态的业务知识文档

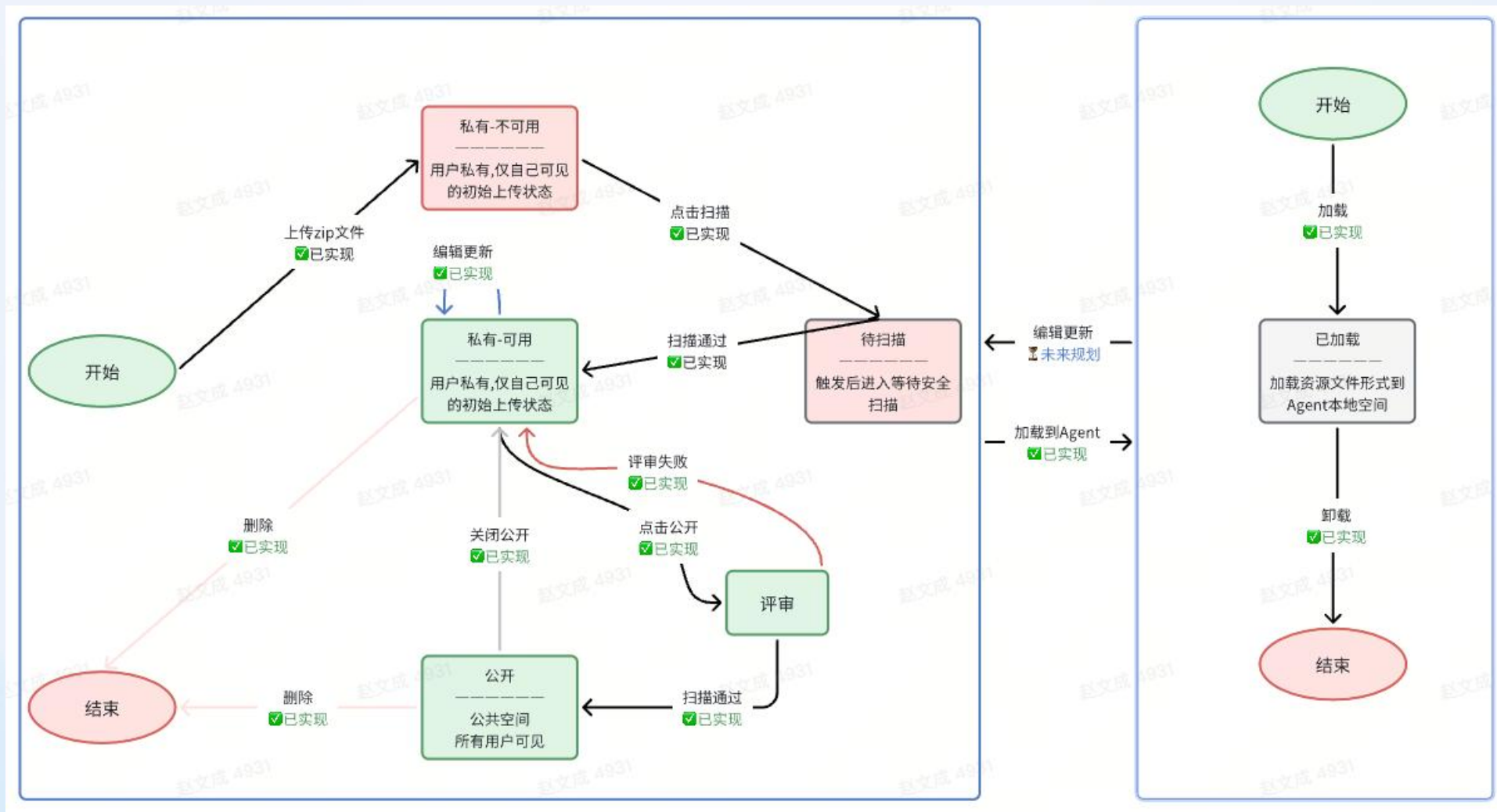
建设思路：文档结构化与场景化复用

记忆 (Memory)

结构化、可关联的场景化上下文与案例。



Skills 原子能力的建设 - 生命周期概览



共识 Skills 创建规范

技能质量

- 最佳实践：是否符合Agent Skills规范
- 完整性：SKILL.md、PRACTICES.md 等是否齐全
- 清晰性：描述是否清晰，指令是否结构化
- 安全性：是否有密钥泄露或危险模式
- 可维护性：结构是否清晰，文档是否完善

命名规范

- 只能包含小写字母、数字、连字符
- 不能以连字符开头和结尾
- 不能包含连续连字符
- 目录名必须与 name 字段一致



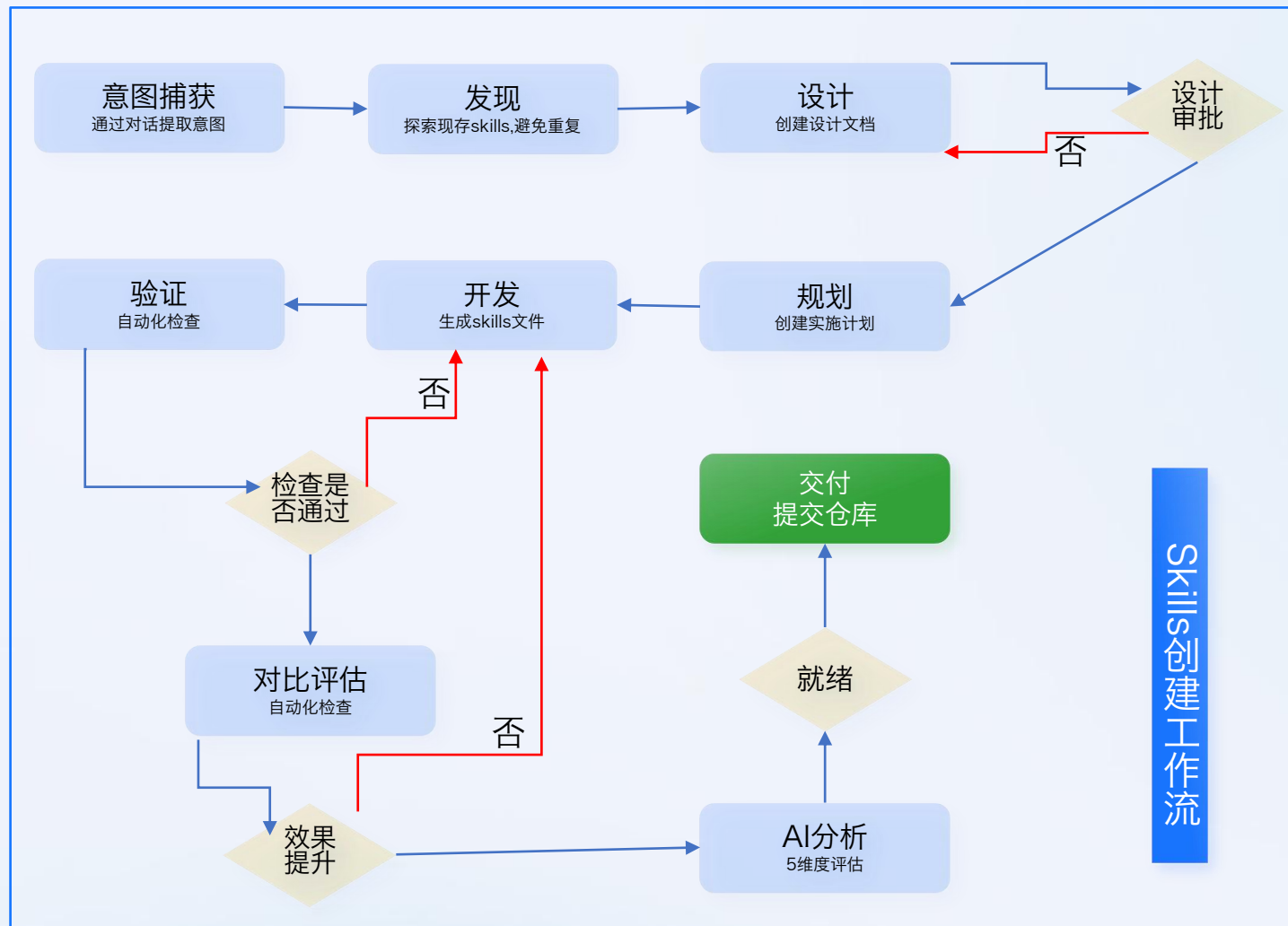
把规范融入到工具

工具核心特征

- AI驱动创建：自然语言对话设计和生成 Skills
- 自动化验证：语法检查、安全扫描、测试执行
- 五维度评估：完整性、清晰性、最佳实践、安全性、可维护性
- CI\CD集成：GitLab 流水线 + 飞书通知

提供统一的创建工具

- 规范生成工具：质量更有保障
- 提供官方Skills：常用系统官方封装



Skills 三类扫描维度

敏感信息扫描

- 云服务凭证 (AWS、阿里云、腾讯云等)
- 代码仓库令牌 (GitHub、GitLab)
- JWT Token、数据库连接字符串

代码安全扫描

- SQL注入、命令注入、路径遍历
- SSRF、XSS风险

危险指令扫描

- 文件删除 (`rm -rf`)
- 权限提升 (`chmod 777`)
- 网络隧道/反弹shell



Skills 技能托管

Skills技能管理

发现并使用更多强大的Skills技能

选择部门

公共技能 个人技能

请输入名称进行搜索

+

新建

上传

公共技能

共 51 条

官方

mi

mi-mcp-skinner-cli

chai

☆ 3

CLI tool skill for API interaction. ALWAYS run `<command> --help` before executing any...

官方

mi

devx

chi

☆ 1

Query Xiaomi DevX 3.0 platform for application management, deployment history, and git repository...

官方

mi

domain

☆ 1

"Xiaomi internal domain management system. Use this skill when you need to query DNS records,...

官方

mi

iam

cun

☆ 1

Query Xiaomi IAM platform for service tree, user permissions, and resource bindings. Invoke this skill...

官方

mi

cun

☆ 1

Query Xiaomi LVS (Load Virtual Server) platform for multi-cloud load balancer resources. Invoke this skill...

官方

mi

matrix

n

☆ 1

Query Xiaomi Matrix container cloud platform. TRIGGER when user mentions: IP address lookup f...

官方

mi

m

c

☆ 1

"Xiaomi internal host control system. Use this skill when you need to query host resources, host...

官方

mi

m

c

☆ 1

Read-only MIFE troubleshooting skill for domain access confirmation, rule lookup, service/backend...

官方

mi

prompter

c

☆ 1

Transform complex natural language requests into executable task plans with dynamic skill...

官方

mi

falcon

c

☆ 0

'Query Xiaomi Falcon monitoring service for endpoints, counters, alert events (via Nox alarm...

mi

fe

l-mcp

☆ 5

访问飞书 (Lark) 生态系统以管理云文档、用户和评论。当用户想要读取、写入、创建或搜索飞书文档...

mi

redis

analysis

☆ 3

Redis 服务器网卡流量分析

mi

domainResolve

we

☆ 1

get domain info of XiaoMi.co.ltd.

mi

ob-scale-in-precheck

☆ 1

查询OB租户是否可以缩容，为发布会大促结束后的缩容和日常租户缩容提供判断依据。使用Prometheus接口...

mi

db-memoryflow

hua

☆ 1

red 工具。

mi

slow_sql_optimize

☆ 1

该技能用于分析数据库慢SQL，获取表结构、执行计划 (EXPLAIN)，并给出SQL性能优化建议。适用于...

mi

"system-monitor"

☆ 1

"获取远程机器的 CPU、内存、IO 信息。Invoke when user provides machine IP/hostname and needs..."

mi

loki-log-cluster-standalone

z

☆ 1

Query Loki logs with async HTTP calls and cluster similar log content using SimHash to surface...

mi

"Acronym_Generator"

v

☆ 1

"Generates optimal English abbreviations and full names from Chinese department names, business..."

mi

Self-Improving + Proactive Agent

☆ 1

"Self-reflection + Self-criticism + Self-learning + Self-organizing memory. Agent evaluates its own..."

知识库 建设思路

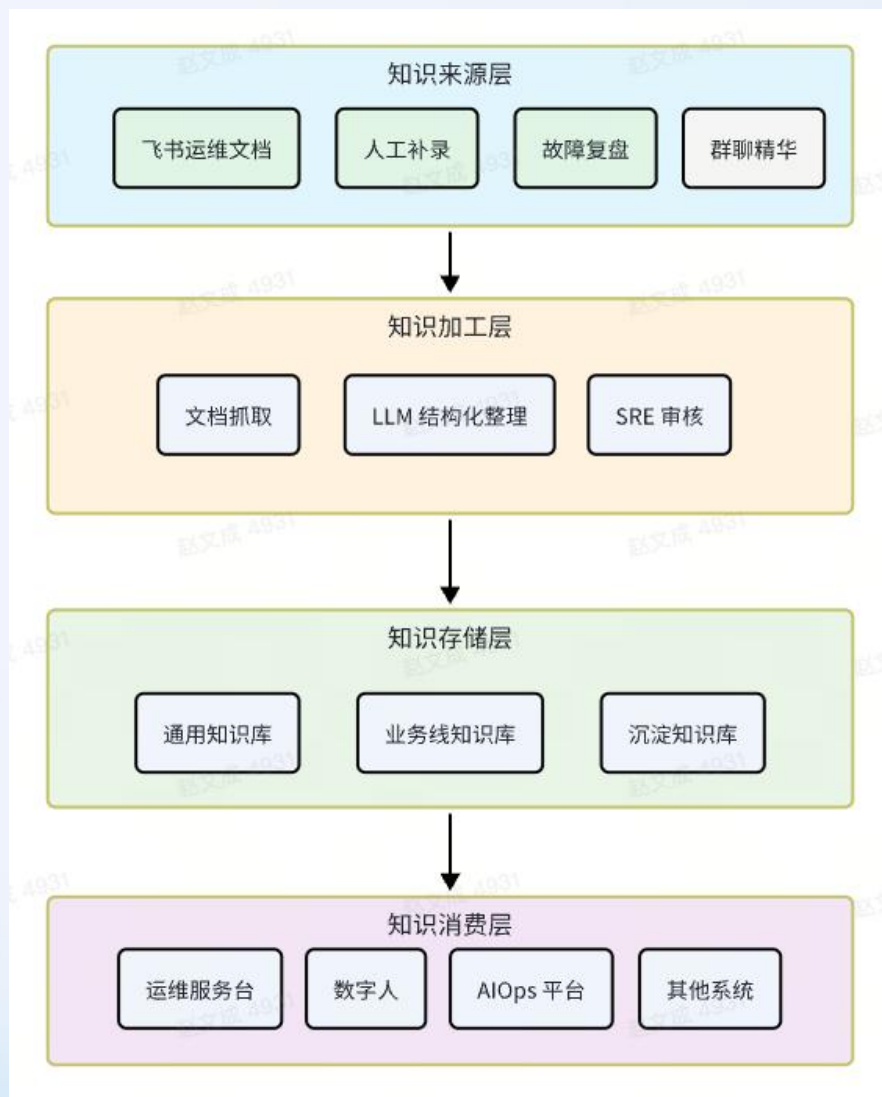
知识库作为独立的基础服务进行建设

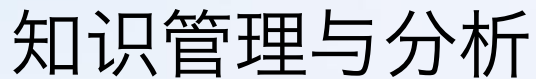
基于飞书文档：将历史沉淀快速复用，只抓取入库不改写原文档

生产与消费解耦：消费方通过API进行消费，无需关心知识生产

宁缺毋滥原则：保证入库质量

知识分层：通用知识、业务线知识、沉淀与抽取





知识管理 & 知识分析

管理：分类分组、查询、批量导入

分析：未命中分析，忽略或者补录

问题：无全局视角查看，过期知识识别困难



记忆维度与探索阶段

记忆维度

- 业务维度：自定义Agent，独立记忆
- 用户维度：不同用户，独立记忆

探索阶段

第一阶段：

- MySQL实现：定时提取，前端展示
- 效果：会误导模型

第二阶段：

- 纯文本方案：标注存储路径，纯提示词，存读模型决定
- 效果：有一定效果，某些场景下链路变长

第三阶段：

- LLM抽取：存向量数据库，自定义抽取逻辑，干预读写
- 效果：复杂，还在演进探索中



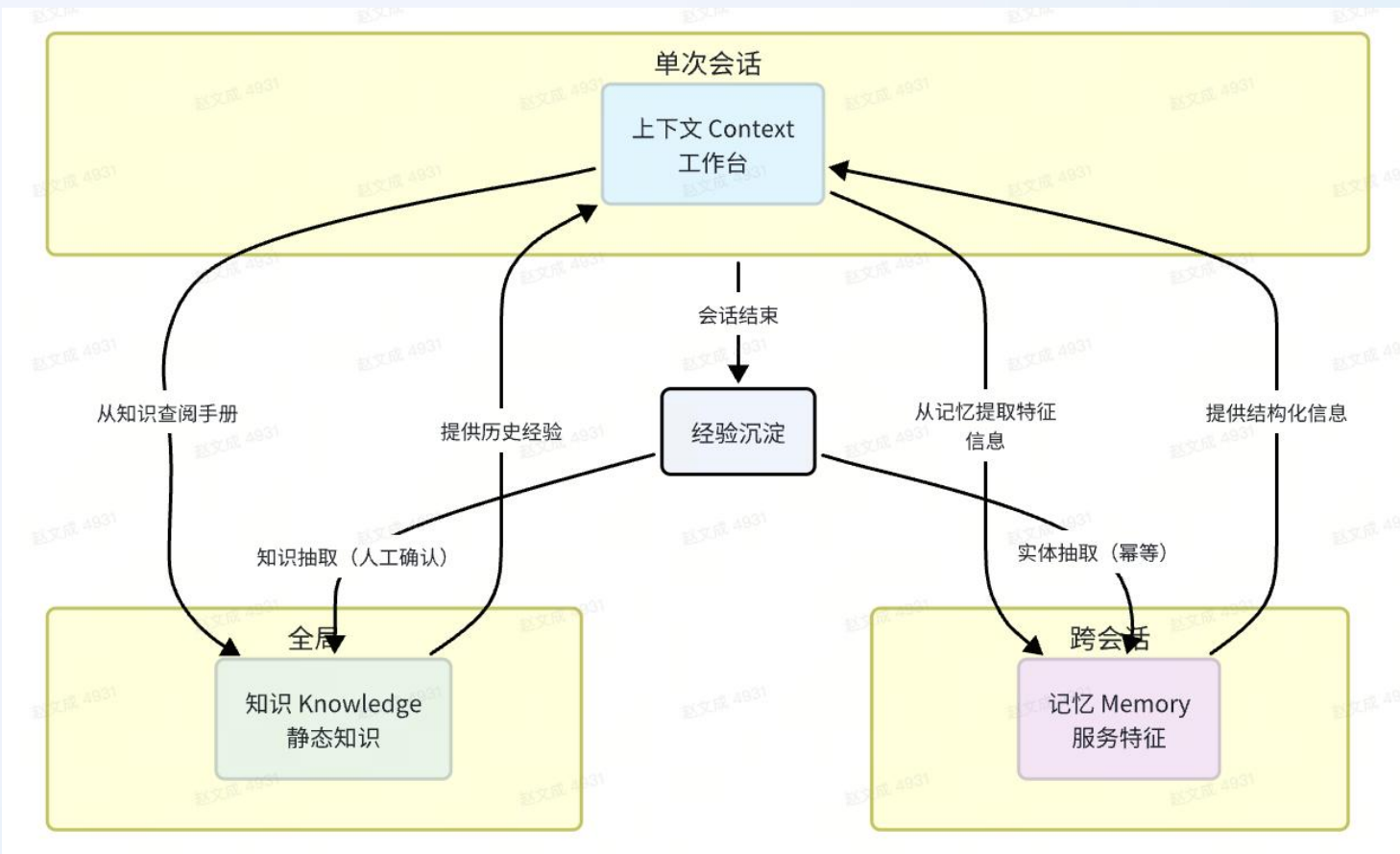
● 记忆+知识 完善业务信息

以业务为维度的“信息”系统

上下文：本次对话中 Agent 需要看到的信息，单次 session 有效

记忆：业务的特征信息，某个服务的上下游信息，跨越会话存在。

知识：持久化信息，业务所在节点，历史故障根因等



03

AIOps平台简介

整体架构介绍

平台功能介绍

AIOps 平台基础能力

智能助手：通用模式对话入口

智能体&应用：自定义 Agent

MCP工具：MCP工具托管

Skills管理：Skills 纳管

任务管理：将多智能体能力对外

执行记录：对话历史与对话结果



平台功能介绍

智能助手

智能体

应用管理

MCP工具

SKILL...

模型

单智能体

多智能体

任务管理

执行记录

智能运维平台

你的AI助理 @小螃蟹 · 在线

性格偏好

你有很多角色，每种角色有不同的要求，会在不同的场景下激活。
以下是几个角色的定义：
名字：SRE工程师
描述：专业站点可靠性工程师，专注于 SLO、错误预算、可观测

任务计划

我的工作

历史会话

04-10 15:19

04-10 15:19

04-09 20:43

04-09 20:43

04-09 20:43

04-09 20:43

04-09 20:43

04-09 20:43

04-09 20:43

04-09 14:46

新会话

你好，我是你的专属AI助手 @小螃蟹

无论工作难题还是生活琐事，我都愿意在这里，用心倾听与回应。

帮我写一份周报

总结本周工作内容和下周计划

服务运维专家

使用域名/LVS/Es/Falcon等工具运维服务

任务

用户空间

输入您的需求...

技能(10)

mimo-v2-pro

发送

任务

用户空间

沙盒

文件

独立沙盒

运行中

.cache

.config

.micode

.venv

memory

daily

输入您的需求...

技能(10)

mimo-v2-pro

发送

任务

用户空间

查询用户 zhangyimin7 的 IAM 节点权限

查询手机、服务器、核心系统、系统框架部下的 IAM 节点

调查 API 的申请流程

任务查看

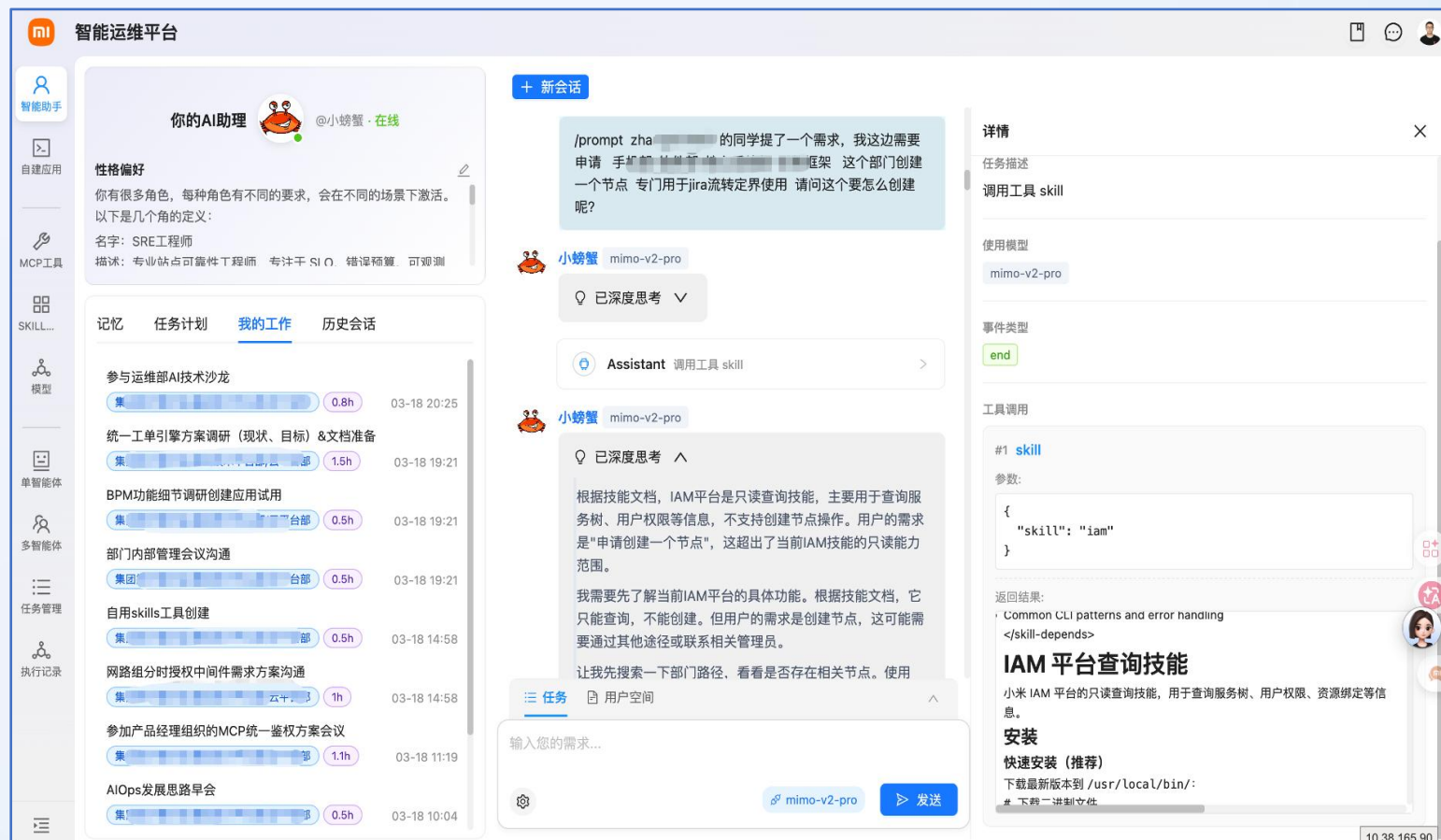
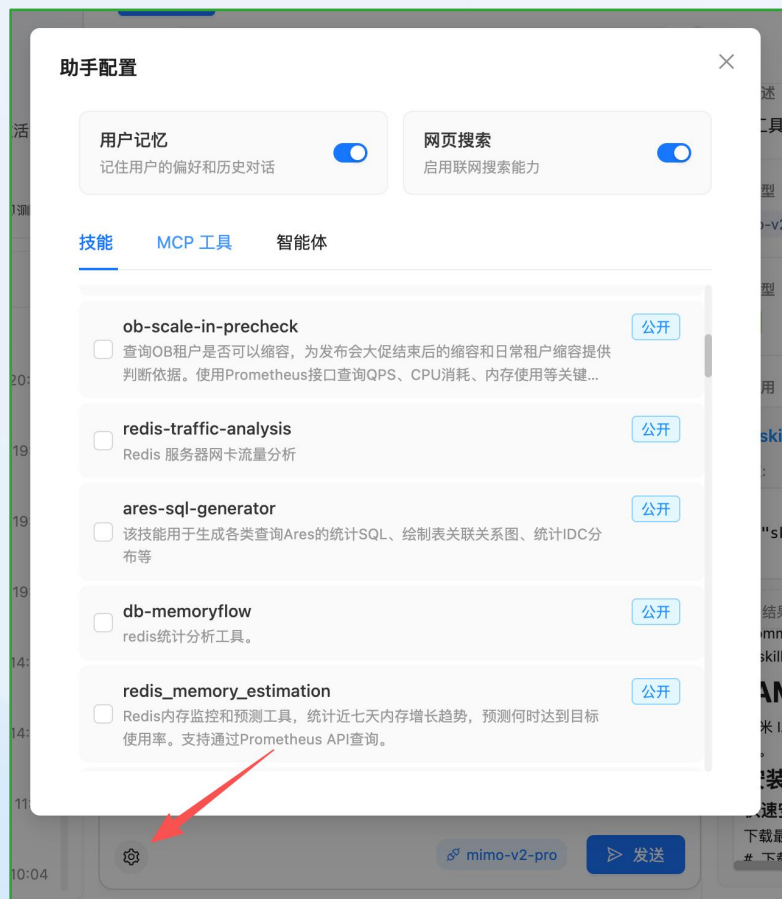
输入您的需求...

技能(10)

mimo-v2-pro

发送

功能概览

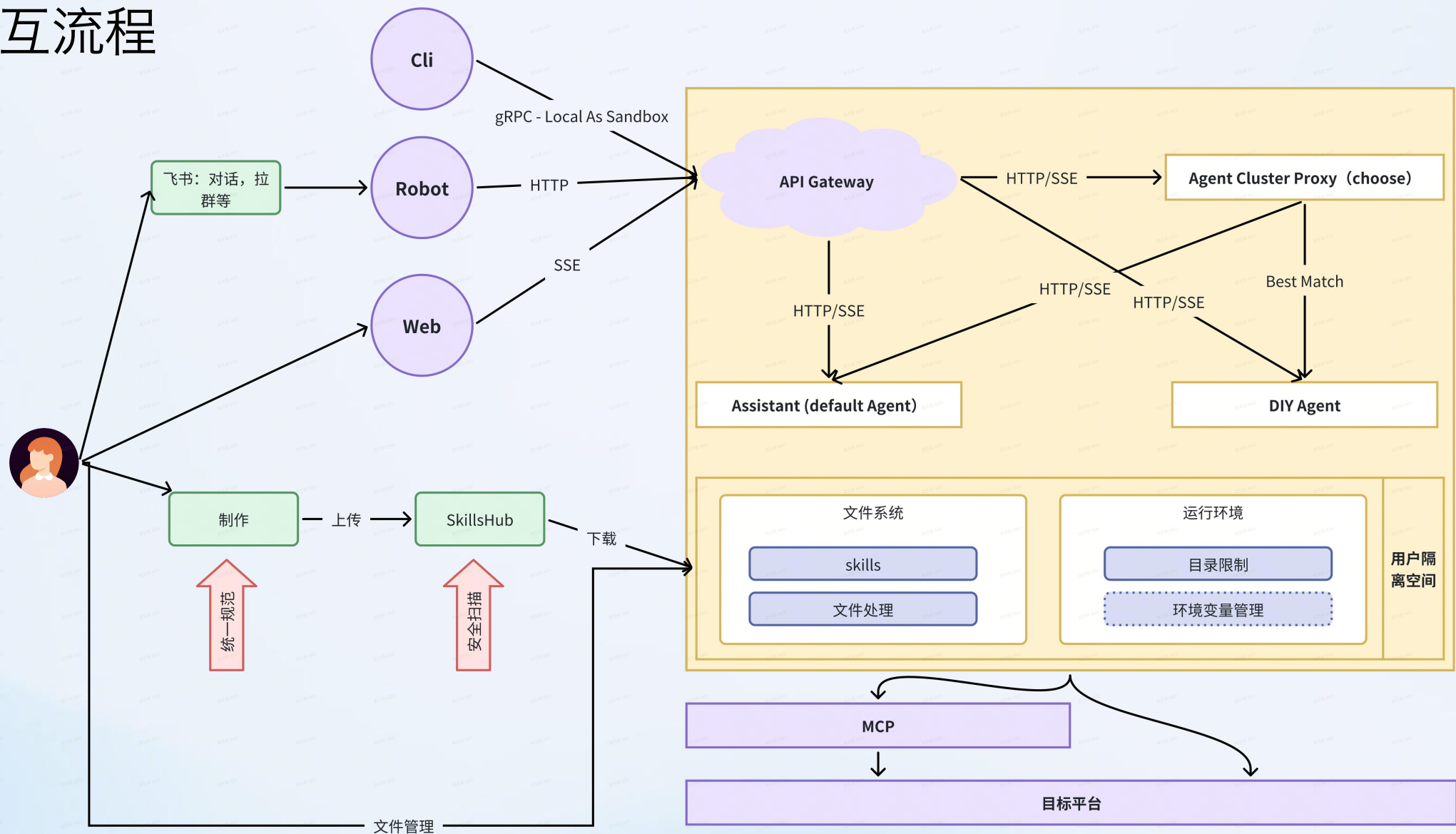


整体架构图





交互流程



支持自定义Agent & 默认Agent

默认Agent

- 通用能力内置
- 开箱即用
- 覆盖常见场景
- 降低使用门槛

门槛低

自定义Agent

- 用户基于不同场景配置
- 统一配置管理
- 灵活性高
- 适配特殊业务需求

更灵活

多维运行策略

独立执行环境

独立交互入口

独立工具集

自定义Agent

IAM-Agent

域名管理-Agent

ITSM系统-Agent

Redis监控-Agent

默认Agent

用户

1. 通过默认Agent交互

2. 直接与自定义Agent交互

可自定义的维度:

- 可用工具集自定义
- 群内Oncall策略自定义
- 记忆、知识库独立
- 一只独立“龙虾”，可以绑定专属的机器人
- 支持被默认Agent唤醒

04

落地案例展示

四类真实场景的落地案例

案例一：通过自定义 Agent 实现告警自动分析-redis



场景：SRE 告警自动分析

告警痛点：SRE 睡不好觉，随时随地出现告警

实现方式：通过自定义 Agent，把AIOps机器人拉入告警接收群，自动监听群告警

特点：指定的告警自动分析





Redis故障分析实践

数据库类型: redis

告警次数: 1

类型: MEM

实例: 10.1.1.1

角色: slave

触发时间: 2026-03-27 20:20:47

IAM树: xi.1.1.1

业务Owner: wur.1.1.1

业务DBA:

Oncall DBA: @1.1.1

详情: 内存使用率: 96.93%, 已使用内存: 1.94G

失败原因: forbidden

扩容失败(huangmingyu)

- 根据配置规则自动监听目标告警
- 获取集群基本信息，包括集群名称，集群架构类型，集群拓扑关系
- 异常指标多维度分析包括内存，CPU,QPS，慢查询数量等等
- 给出根因分析并提出处理建议
- 点击“查看详情”打开侧边栏可以继续追问

Redis集群故障分析报告

集群信息

- 集群名称: cn_
- 架构: Redis 12 (1分片, 1主1从)
- 异常实例:
 - Master: 10 37
 - Slave: 10 5
- 触发时间: 2026-03-27 20:20:47

⚠ 异常指标 (共1项)

- **内存使用率:**
 - Master: **96.58%** (阈值: >90% Critical)
 - Slave: **96.56%** (阈值: >90% Critical)
 - **状态: 严重超标, 22分钟内从~11%急速上升至96%+**

相关指标趋势

- CPU使用率: Master 14.2% (↑164%), Slave 8.2% (↑163%) - 轻度上升
- QPS: Master 7122 (↑148%) - 流量明显增长
- 慢查询: 0条 - 无慢查询问题

根因分析

1. **直接原因:** 内存使用率在20:03-20:25期间急速上升, 22分钟内从11%飙升至96%+, 触发PO告警。
2. **流量因素:** 同期QPS从313增长至7122 (峰值10872), 表明有大量写入操作。
3. **配置变更:** 日志显示20:21:40有 `CONFIG REWRITE` 成功执行, 需确认是否修改了内存相关配置。
4. **架构特点:** 该集群为1片3节点 (主从), 总内存容量有限 (约2GB), 对突发写入敏感。

 紧急处理建议

1. 立即扩容: 由于内存已超96%, 建议立即进行实例内存扩容, 避免OOM导致服务中断。
2. 流量控制: 检查业务写入模式, 如有异常高峰需与业务方协调限流。
3. 大Key排查: 执行 `redis-cli --bigkeys` 命令, 识别并优化大Key, 减少内存碎片。
4. 配置审计: 核实 `CONFIG REWRITE` 操作内容, 确认 `maxmemory` 等关键配置未被意外修改。

 监控快照

- Master内存趋势: [查看图表](#)
- Slave内存趋势: [查看图表](#)

🕒 后续跟进

建议业务方配合分析20:03前后的写入模式，确认内存增长是否由预期内的数据加载导致，并制定长期容量规划。

3. 查询CI/CD系统的配置变更记录

2. 应用层追踪:

1. 检查业务应用日志，寻找配置变更调用
2. 确认是否有自动化脚本或定时任务触发

3. 网络层分析:

1. 根据客户端IP  定位具体应用服务器
2. 检查代理层访问日志

4. Redis连接信息:

当前无法直接执行，建议运维人员检查

```
redis-cli -h
```

⚡ 快速行动建议

1. **立即询问：**联系业务Owner wun [redacted] i，确认是否有计划内的配置变更
2. **检查时间窗口：**20:21:40前后的操作记录
3. **重点关注：**配置变更可能影响了 m [redacted] 等关键参数，导致内存使用率异常上升

由于Redis设计上不记录操作审计信息，建议企业建立完善的Redis操作审计流程，通过代理层日志或专用审计系统记录所有配置变更操作。

三 任务 用户空间

输入您的需求...

➤ 发送

案例一：通过自定义 Agent 实现告警自动分析-重点业务

[故障][P1] [网关]按主机可用性低于 98%

最近一次告警值: 0.03
生效集群: 公共集群
模版名称: itsm 网关告警
标签: instance=ak-itsr-om:9090
开始时间: 2026-03-25 21:21:32
报警持续时长: 10s
告警描述: <no value>
21:22
视图 详情 合并报警>> 屏蔽更多>>
处理人:
认领 屏蔽选项

回复 Falcon报警: [故障][P1] [ITSM][网关]按主机可用性低于 98%

总结

问题: ITSM网关可用性告警, 错误率3%超过阈值2%, 触发P1级别告警。

原因分析:
1. 网关层出现500错误, 主要集中在变更工单审核和变更策略版本接口
2. 后端实例未发现异常日志, 问题可能集中在网关层
3. 特定API请求处理失败导致整体可用性下降
解决方案:
1. 立即检查相关API接口的后端服务状态和网关配置
2. 联系SRE团队()进行故障排查
3. 优化网关错误处理和监控机制, 防止类似问题再次发生
SRE信息: 域名 proc-om-et 的SRE团队
查看详情

告警信息: 5xx, 时间 10:31 开始。
Step 1: 域名信息查询
域名查询到了! 再查解析记录:
Step 2: ES 日志查询
ES 查询超时了, 试试具体 URL 查询:
日志数据非常详细, 继续 Step 3: IAM 查询
Step 3: IAM 查询
所有数据都拿到了。现在汇总输出完整排查报告:
域名告警排查报告 - kt-com
1. 告警概要
告警类型: [恢复][P1] 5xx 状态码错误
告警时间: 2026-03-26 10:31:08, 持续 24 分钟后恢复
异常服务: xiaomi-
异常域名: k- (售后知识库)
2. 根因定位
异常模式: 模式C - URI集中 + 代理层主机分散 + 上游集中
根因结论: 后端服务异常 (单一上游故障)
3. 证据分析
3.1 代理层主机分布 - 分散
15+ 个 Mife 节点均有 5xx, 单节点最多 87 个 (fe06), 没有单一节点集中异常。
3.2 上游服务分布 - 集中
所有 1988 个 5xx 请求全部指向同一个上游: 10.:8080
这是关键证据 - 说明不是 Mife 层问题, 而是后端服务 10.:8080 故障。
3.3 URI 分布 - 集中在 5 个接口
URI 5xx 数 占比
/api/thread/getThreadTag 432 21.7%
/api/thread/suggestSearch 195 9.8%
/api/search/querySuggestList 134 6.7%
/api/kbsTrack 127 6.4%
/api/thread/getRelatedLore 80 4.0%
1/2
3.4 Status Code 分布
502: 1622 个 (81.6%) - 上游无响应或连接被拒
504: 366 个 (18.4%) - 上游超时
3.5 客户端 IP 分布 - 分散
Top IP 来自外部用户 (124.239.x.x, 117.159.x.x), 非攻击特征。

3.4 Status Code 分布
502: 1622 个 (81.6%) - 上游无响应或连接被拒
504: 366 个 (18.4%) - 上游超时
3.5 客户端 IP 分布 - 分散
Top IP 来自外部用户 (124.239.x.x, 117.159.x.x), 非攻击特征。
3.6 请求耗时
p50: 3.04s (正常应该 <500ms)
p95: 120s (几乎等于超时上限)
上游耗时 p95 同样 120s, 确认瓶颈在后端
4. 域名信息
真实域名: k-om
用途: 售后知识库
域名 Owner: zha-1
申请人: m-ng
所属 IAM 节点: xiaomi-infomms (售后组)
证书到期: 2 天后
CDN: 有 (指向 n-om, 当前生效中)
DNS 解析: CNAME -> kbs.k-mi-dun.com
5. 人员信息
角色 姓名 邮箱
SRE Leader 宋- so-i.com
SRE We- (ing) fer-mi.com
SRE 周- z-omi.com - 告警处理人
SRE Jc- jc-omi.com
Owner 范- (dong) fa-i.com
1/2
6. 结论
上游单一节点 10.:8080 故障, 导致 Mife 代理层返回 502/504。代理层本身无异常, 问题定位到后端 kfs-atform 服务。已恢复, 建议排查该上游节点的健康检查和故障自动摘除机制。
7 步 SOP 完整跑通 5 个 MCP Server 串联, 从告警到根因定位全自动。这就是以后报警群里的标准排查流程
已完成 耗时 1m 49s

案例二：结合“知识”体系实现群内 Oncall

场景：群内自动 Oncall

Oncall 痛点：Oncall 信息繁琐，影响SRE效率

实现效果：帮助 SRE 在群内与研发沟通，回答针对此服务的问题，同时也可以完成指定告警分析

特点：根据对话上下文进行知识抽取与存储



OSMOS 3月16日 17:04

回复 @OSMOS 但是get

by

分析报告

结论: 你说得对——从本次复核结果看, g 在该时段并没有报错/也没对应到 hlth e 超时; 这次告警实际是 /a, failrate 暴涨 触发, 同时 /he e 指标冲高也在同一告警规则覆盖范围内, 但不是同一个 path。

依据

- 按告警覆盖的 5 个 path 逐个复核 (21:50-22:10):
 - /app ist failrate 峰值=540 (明显异常)
 - /t id skipfailrate 峰值=10000 (指标冲高)
 - /app ry fail e 均为 0
- 日志复核 (hlth -22:10): 未检索到 get ory 相关错误关键字 (为空结果)。
- 日志中 hlth_center_data_source 超时对应的上层 path 主要是: /t _info、/hea d_url; 9988(token exhausted / connect refused) 主要映射到: /heal _info、/app/v1/stat _y_time ——均不包含 get ry。

建议

- 聚焦 get_s st 的失败原因: 需要把 /app/v1/ list 的错误在日志里“按 path 定位”(目前文档给的 Loki 模板只能固定选 err+SESSION, 缺少标准的二次过滤写法)。请你给一个该接口失败请求的 traceid (或业务侧标准“按 path 二次过滤”语法/面板), 我让 LogAnalyst 直接把 Top 错误栈对齐到该接口。
- 同时确认 /he bind 的 skipfailrate=10000 是否为指标含义/打点异常 (例如 skipfailrate 的单位/倍率问题), 避免和真实 failrate 混淆导致告警解读偏差。

3月16日 17:05

回复 OSMOS: 分析报告

@OSMOS 分析为什么depl 是hlth , 而不是hlthpr、hlth ?

OSMOS

诊断经验沉淀确认

AI 提取了以下知识, 请确认是否入库

提取的诊断经验:

hlth 5 错误率飙升: 下游 gRPC 连接拒绝导致级联失败

hlth 5 在 20:15 出现滚动式错误率尖峰 (错误率最高约 64.19%), 日志集中出现 gRPC 拨号失败: dial tcp 10.1.1.2: connect: connection refused, 影响 60% pod。该模式表明下游 gRPC 服务端不可用/进程未监听/被重启或崩溃, 导致调用方大量连接被拒绝并触发级联失败; 随后在 21:06 触发负载均衡自动摘流, 带来多实例 QPS 断崖式下跌。修复/缓解措施: 立即检查 10.1.1.2 对应服务存活与端口监听状态, 核对 20:15 是否存在发布/重启等变更; 若确认下游故障, 回滚下游版本或启用降级策略以恢复上游稳定性。

适用场景: 调用方服务出现大面积 gRPC 错误率短时暴涨并伴随 LB 自动摘流、QPS 急跌时

确认入库

丢弃

3月16日 17:12

回复 OSMOS: 分析报告

@OSMOS 我告诉你结论吧, hlth oc调用data server出现了超时, 时间是1500ms, 你从日志里也能看到。下游data 并未超过1500ms, 所以这个问题基本可以认为是网络抖动等超时导致的整体失败。你记一下这个结论, 下次同样的问题可以从这里切入, 放到你的知识库中

OSMOS

抽取处理经验并落地知识库

入口审批前人工确认正确性

诊断经验 — 已丢弃

已丢弃

hlth 5: /app ist source_list 在 21:50 failrate 持续高位触发 P0 (c5 显著高于 c3/ak)

同一告警规则覆盖多个 path 时, 需逐个 path 复核确认真实触发源。本次实际触发源是 hlth 5 的 /app pr ory 在该时段 failrate 为 0, 并非告警根因。depl ace 之所以命中 c5, 是因为该窗口内 get_source_list 在 c5 的 failrate 明显更高且更持续 (c5 Max=80.05 Avg=34.01), 而 c3 基本正常 (Max=1.77 Avg=0.08), ak 只有短促尖峰未形成持续触发 (Max=80.05 Avg=34.01)。

适用场景: 适用于同一告警规则聚合多个接口/路径、且需要解释为何只在某个 deploy_space 触发的场景; 以 2 时间窗对比各 depl 统计值来定位触发源与差异面。

OSMOS 3月16日 17:13

回复 @OSMOS 我告诉你结论吧, hlth c...

分析报告

结论: 已将你提供的 RCA 结论沉淀到知识库: hlth_center_data_source_server 的 gRPC 1500ms DeadlineExceeded, 且下游自身耗时未超 1500ms 时, 优先按网络抖动/链路超时方向排查。

依据

- 关键特征: hlth c 日志出现大量 connect deadline exceeded 且 cost 1500ms (客户端 deadline 打满)。
- 下游 data 侧并无对应的 >1500ms 处理耗时, 说明更可能是端到端链路 (网络/连接/排队/sidecar) 问题而非下游业务慢。

建议

- 下次同类告警的排查切入点: 先统计 DeadlineExceeded 占比与涉及接口 → 对齐是否集中在某 depl space/机/节点 → 联动网络/连接指标与 sidecar/连接池, 再决定是否临时放宽 depl 或做降级。



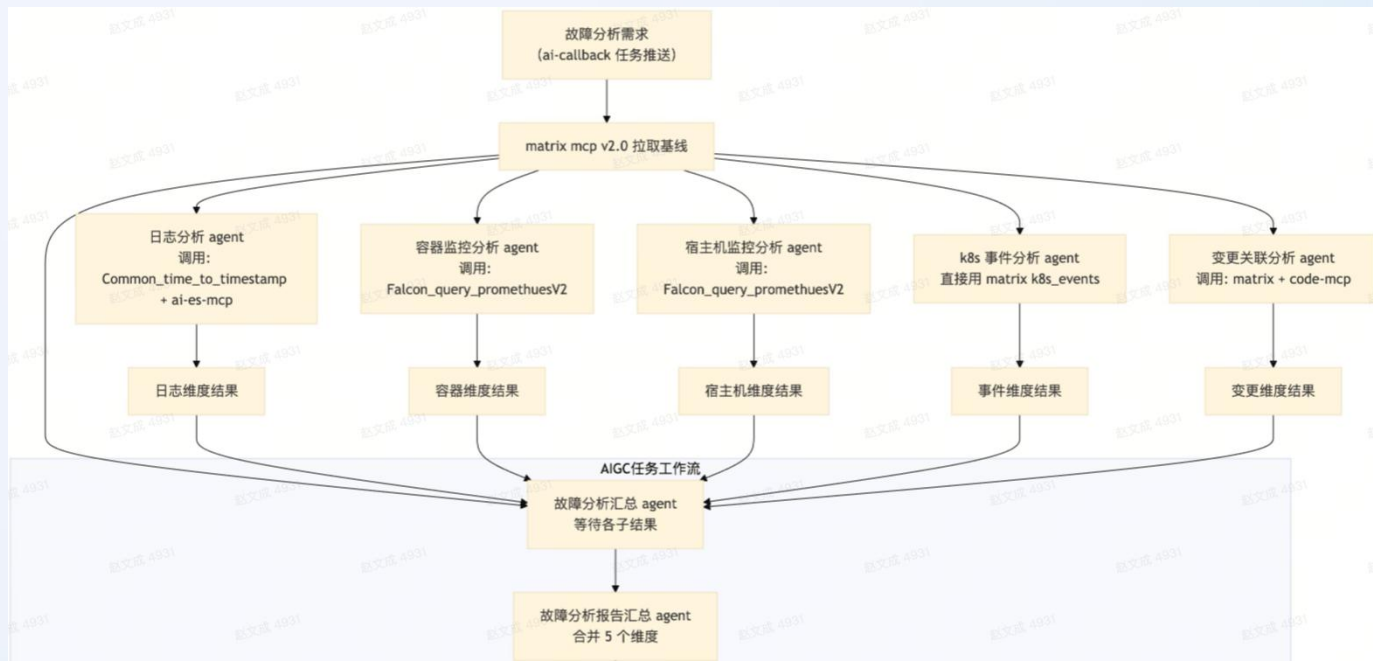
案例三：容器单点异常排查

场景：针对单点异常进行排查与处理

单点异常定义：某个容器服务的个别 Pod 异常，重建后服务可迅速恢复

痛点：出现异常，随时随地需要拿出电脑，分析判断是否是单点异常，半夜或者通勤路上效率很低

解决：自定义应用，出现异常先分析是否为单点异常，手机侧直接可查结果，可操作删除异常Pod，快速恢复业务



案例三：容器单点异常排查

SRERobot 机器人 | SRE智能小助手 16:21

【ai-callback】reco[redacted]er服务疑似单点异常
模板组: [redacted]vice

判断规则: 线上容器服务main容器CPU使用率大于80%
部署单元: reco[redacted]er-c3
IAM节点: xiaomi.[redacted]er 【3[redacted]5】
异常实例: recomme[redacted]c3-gr56w
异常实例所在宿主机: c[redacted]01.bj
异常实例版本: v54
异常实例状态: 正常
集群Running实例占比: 100% (7/7)
灰度状态: 无灰度, 最新版本v54, 实例数7
最近部署人: y[redacted]
最新变更时间: 2025-12-02T09:52:44+08:00
最新变更详情: feat: Upload api
建议: 关注集群状态和灰度发版情况, 建议yan[redacted]排查&删除实例
AIOPS故障分析: AIOPS任务中心 ([redacted]中, 耐心等待三分钟)
[监控链接](#) [业务监控大盘](#) [matrix链接](#)

一键jdump 实例摘流(业... 实例摘流(mis...)

强行删除实例

SRERobot 机器人 | SRE智能小助手

【ai-callback】reco[redacted]er异常实例已手动删除成功

处理方式: 手动删除
处理人: g[redacted]
处理时间: 2025-12-08T16:35:14.545298
处理结果: 手动删除成功

✅ 实例信息快照
判断规则: reco[redacted]56w
部署单元: reco[redacted]3
IAM节点: xiao[redacted]er 【370515】
异常实例: reco[redacted]6w
异常实例所在宿主机: c3-se[redacted]bj
异常实例版本: v54
异常实例状态: 正常
集群Running实例占比: 100% (7/7)
灰度状态: 无灰度, 最新版本v54, 实例数7
最近部署人: [redacted]
最新变更时间: 2025-12-02T09:52:44+08:00
最新变更详情: feat: Upload api
建议: 关注集群状态和灰度发版情况, 建议 @[redacted]排查&删除实例

[监控链接](#) [业务监控大盘](#) [matrix链接](#) [Alops链接](#)

案例三：容器单点异常排查

平台可以查看历史执行记录与报告

The screenshot displays the '智能运维平台' (Intelligent Operations Platform) interface. The top section shows a list of historical execution records with columns for Name, Type, Triggerer, Trigger Time, Record ID, Execution Status, and Action. Below this, the '报告' (Report) section is visible, showing a detailed analysis report for a container node failure. The report includes sections for Overview, Discovery, Container/Pod Monitoring, Host Monitoring, Change Correlation, K8s Events, Application Logs, Initial Conclusion, and Recommendations. The right side of the interface shows the '会话记录' (Conversation Record) section, which includes a '规划清单' (Planning List) and a '派发' (Dispatch) section.

名称	类型	触发人	触发时间	记录ID	执行状态	操作
通用域名异常定位	多Agent	pr...	2026-04-11 07:06:39	e...	已完成	查看 结论
通用域名异常定位	多Agent	pr...	2026-04-11 07:06:09	...	已完成	查看 结论
通用域名异常定位	多Agent	pr...	2026-04-11 07:05:43	4...	已完成	查看 结论
[ai-callback]综合故障分析	多Agent	...	2026-04-10 19:09:52	1...	已完成	查看 结论
[ai-callback]综合故障分析	多Agent	...	2026-04-10 18:29:47	bf9...	已完成	查看 结论
通用域名异常定位	多Agent	...	2026-04-10 11:10:20	...	已完成	查看 结论
通用域名异常定位	多Agent	pr...	2026-04-09 12:16:47	...	已完成	查看 结论
通用域名异常定位	多Agent	pr...	2026-04-09 05:33:54	...	已完成	查看 结论

故障分析报告

概览
Pod `iot-...` (主机 `c3-se...`)

发现

容器/Pod监控

- CPU使用率正常 (平均3.0%, 峰值24.5%), 内存使用稳定 (RSS平均64.9%)
- 与同组Pod指标一致, 无资源瓶颈

宿主机监控

- 系统负载在11:03出现峰值105.2 (阈值192), CPU使用率同步上升至59%
- 内存 (47%) 和磁盘I/O (平均3.4ms) 正常, 资源充足

变更关联

- 当前版本v131 (commit d402fac1) 为刚发布版本, 与Pod启动时间高度接近
- 主要变更: 修复动态信号查询语法 (ToSpeak...ala逻辑调整)

K8s事件

- 无异常事件记录, Pod稳定运行近1天 (启动时间2026-01-05 16:44:05)

应用日志

- 3条WARN日志 (无ERROR): TraceAc...abilityActor超时 (traceId: d6b37c...)

初步结论

1. 系统资源层面无故障, 宿主机负载峰值未超阈值但需关注;
2. 版本变更 (v131) 灰度发布与Pod启动时间重合, 存在关联嫌疑;
3. 业务链路超时警告提示潜在API调用问题。

建议

****业务/研发**:**

1. 立即对比v131版本其他实例状态, 确认超时问题是否普遍 (operator跟进);

规划清单

- [ai-callback]matrix容器节点pod故障分析助手
- [ai-callback]matrix宿主机故障分析助手
- [ai-callback]变更关联故障分析助手
- [ai-callback]k8s事件分析助手
- [ai-callback]小夏日志分析助手
- [ai-callback]报告汇总

派发 [ai-callback]matrix容器节点pod故障分析助手

[[ai-callback]matrix容器节点pod故障分析助手] 分析 iot-agent-skill-prev-4xrp Pod 在 iot-agent-skill-prev 部署空间的容器监控数据, 识别 CPU、内存、网络、磁盘等指标异常情况, 时间范围 now-5m 到 now

子Agent最终响应 [ai-callback]matrix容器节点pod故障分析助手

[[ai-callback]matrix容器节点pod故障分析助手] 分析 iot-agent-skill-prev-4xrp Pod 在 iot-agent-skill-prev 部署空间的容器监控数据, 识别 CPU、内存、网络、磁盘等指标异常情况, 时间范围 now-5m 到 now)

案例四：SRE 综合性问题查询

SRE 运营类知识查询

- 帮我查询一下 mi.com 的子域名有哪些？需要对国内的子域名进行治理，请分析有多少个域名要治理

现在让我给用户一个完整的回复，总结我的发现和建议。

根据域名管理系统查询结果，我为您完成了 **mi.com** 子域名的治理分析。

查询结果摘要

mi.com 子域名总数：11 个（不包括 **mi.com** 顶级域名本身）

按环境分类统计：

环境类型	数量	说明
生产环境 (product/production)	1 个	需要优先治理
测试环境 (test/staging)	3 个	可后期处理
预发布环境 (preview)	1 个	可后期处理
未标记环境	1 个	需要进一步确认

按业务模式分类：

- 测试环境 (test-staging-mi.com): 3 个
- 预发布环境 (preview-mi.com): 1 个
- 活动域名 (activity-mi.com): 1 个
- 订单相关 (*-order-mi.com): 1 个
- 后台管理 (*-admin-mi.com): 1 个
- 高清活动 (hd-mi.com): 1 个
- 其他: 35 个

关键生产环境域名 (需优先治理)

- mi.com - 小米官网 (最高优先级)
- mi.com - PC 官网活动域名
- mi.com - 订单系统
- mi.com - 购物车
- search-mi.com - 搜索服务
- api-mi.com - 订单 API
- service-mi.com - 售后服务
- static-mi.com - 静态资源
- q-mi.com - 企业采购
- h-mi.com - 人力资源

治理建议

短期 (1-2周):

- 盘点确认：核实每个域名的实际使用情况
- 优先级排序：按业务重要性分级处理
- 制定迁移方案：为关键域名准备对应替代方案

中期 (1-2个月):

- 域名映射：创建对应替代域名
 - 例如：www-xxxxx.com
 - 例如：order-xxxxx.com
- 配置更新：更新 DNS、SSL 证书和应用配置
- 测试验证：在测试环境验证迁移方案

长期 (3-6个月):

- 流量迁移：设置 HTTP 重定向，逐步切换流量
- 监控优化：监控迁移过程，优化用户体验
- 清理回收：停用旧域名，更新内部文档

任务 用户空间

案例五：SRE 综合性问题查询

Assistant mimo-v2-pro

Q 已深度思考

现在我已经了解了 Matrix 和 IAM skill 的能力。让我也检查一下是否有 MCP 工具可用。根据 prompter skill 的说明，我需要使

根据系统提示，我有以下工具可用：cloud_knowledge_get_detail, feishu_notify_safe, user_completed_work, user_crontab, user_crontab_del, now, feishu_authorize, feishu_search_doc, feishu_fetch_doc, feishu_create_doc, feishu_update_doc, feishu_list_docs, write_todos, task, skill, ls, read_file, write_file, edit_file, glob, grep, execute。

我没有 ListMcpResourcesTool。不过，根据系统提示，有提到“官方的MCP服务被默认绑定在了mcp_router工具下提供渐进式加载”，但我没有看到 mcp_router 工具。这可能意味着 MCP 工具可能不可用，或者我需要通过其他方式发现它们。

让我先分析一下用户的需求。用户想要分析 xiaomi.miliao.account 产品线下的所有 Matrix 的资源利用率和容器规格，并提出改进优化建议。

根据 Matrix skill 的说明，我需要：

- 通过 IAM skill 获取 tree_id
- 使用 Matrix skill 查询资源使用情况

具体步骤：

- 使用 IAM skill 的 search-tree --query "xiaomi.miliao.account" 获取 tree_id
- 使用 Matrix skill 的 get-resource-summary --x-iam-tree-id <tree_id> 获取资源使用率总览
- 使用 Matrix skill 的 list-resource-details --x-iam-tree-id <tree_id> --tree-id <tree_id> 获取详细的资源使用情况

但是，用户还要求“容器规格”信息。从 Matrix skill 的文档中，我看到了 list-pods 命令可以列出 Pod 的详细信息，这可能包含容器规格。此外，可能还需要查看部署单元的设置，以了解资源请求和限制。

让我先制定一个执行计划。

首先，我需要确认这些命令的具体参数。我应该先运行 --help 来查看帮助信息，但根据 prompter skill 的说明，我需要先制定计划并获得用户批准，然后再执行。所以，我现在应该生成一个执行计划。

让我整理一下执行计划：

目标：分析 xiaomi.miliao.account 产品线下的所有 Matrix 的资源利用率和容器规格，并提出改进优化建议。

所需技能：

- iam：获取 tree_id
- matrix：查询容器资源使用情况和规格

执行流程：

- 通过 IAM skill 获取 xiaomi.miliao.account 的 tree_id
- 并行查询：
 - 获取资源使用率总览 (get-resource-summary)
 - 获取资源使用详情 (list-resource-details)
 - 获取部署单元列表 (list-deploy-units)，然后获取每个部署单元的 Pod 列表 (list-pods)，以获取容器规格
- 分析数据并生成优化建议

步骤：

Step 1: 获取服务树 ID

- Skill: iam
- Command: mi-mcp-skinner-cli --spec iam.json --signer iam search-tree --query "xiaomi.miliao.account"
- Output: tree_id

Step 2: 并行查询资源信息

- Step 2a: 获取资源使用率总览
 - Skill: matrix
 - Command: mi-mcp-skinner-cli --spec matrix.json --signer iam get-resource-summary --x-iam-tree-id <tree_id>
 - Dependencies: Step 1
- Step 2b: 获取资源使用详情
 - Skill: matrix
 - Command: mi-mcp-skinner-cli --spec matrix.json --signer iam list-resource-details --x-iam-tree-id <tree_id> --tree-id <tree_id>
 - Dependencies: Step 1

Step 3: 获取容器规格信息

- Step 3a: 获取部署单元列表

- 获取了 xiaomi.miliao.account 产品线的 tree_id
- 获取了所有部署空间的资源详细信息
- 分析了资源使用率和容器规格
- 生成了详细的优化建议报告

现在我要回复用户，提供分析结果和优化建议。让我创建一个简洁的回复。

分析完成！以下是 xiaomi.miliao.account 产品线的 Matrix 资源分析结果：

核心数据概览

- 部署空间：1个
- 总应用数：1个
- 月总成本：¥1,200.00
- CPU 总使用率：100% (生产)
- 内存总使用率：80% (正常)
- 资源不达标率：75%

主要问题发现

- CPU 资源严重闲置：部署空间 CPU 使用率仅为 100%，远低于 100% 的规格限制。
- 规格配置过大：部署空间 CPU 规格限制为 100%，远高于实际使用需求。
- 零使用资源：部署空间完全无使用，但仍产生成本。
- 成本集中度高：Top 10 高成本部署单元占总成本的 50% 以上。

优化潜力

- 预估月度节省：¥1,200.00 (占总成本 99%)
- 年度节省潜力：约 ¥14,400.00

Top 5 优化重点

- 部署空间 (生产环境)：月成本 ¥1,200.00，CPU 使用率仅 100%
- 部署空间 (生产环境)：月成本 ¥600.00，CPU 使用率仅 100%
- 部署空间 (生产环境)：月成本 ¥400.00，CPU 使用率仅 100%
- 部署空间 (生产环境)：月成本 ¥50.00，CPU 使用率仅 100%
- 部署空间 (生产环境)：月成本 ¥50.00，CPU 使用率仅 0.7%

优化建议

紧急行动 (1周内)

- 清理闲置资源：评估并下线无使用部署空间
- 优化高成本资源：调整 Top 5 高成本部署空间的规格

短期优化 (1个月内)

- CPU 规格调整：将规格过大的部署空间 CPU 规格调整到合理水平
- 内存规格优化：优化内存规格过大的部署空间

长期策略

- 建立资源配额管理：按应用、环境设置资源配额
- 实施自动伸缩：为关键服务配置基于指标的自动伸缩
- 建立成本监控：实时监控资源使用和成本异常

生成文件

- 详细分析报告：matrix_resource_analysis_report.md

任务 用户空间

- 通过 IAM 技能获取 xiaomi.miliao.account 产品线的 tree_id
- 使用 Matrix 技能获取产品线下的所有应用列表
- 遍历每个应用获取部署空间和部署单元
- 获取每个部署单元的资源使用率和容器规格

SRE 运营类知识查询

➤ /prompt 帮我分析一下 xxxx 产品线下面的所有 Matrix 的资源利用率和容器规格，并提出改进优化建议

05

经验总结与展望

总结经验与短期规划

总结：技术选型-三种架构演进的取舍

基础模型 + 自主开发（第一版）

- 路径：手写所有业务逻辑和工具调用代码
- 特点：最灵活，不受框架限制，但调试难度高
- 适用：MVP，极简场景

踩坑：

- 手写循环调用、状态管理、工具解析极易出错。特别是流式输出的兼容性问题，会消耗大量非核心业务时间。

现成Harness为执行端（第二版）

- 路径：直接采用现成的Harness（如各种Code）作为执行端。
- 特点：上线速度最快，开箱即用，调试难度极高
- 适用：通用编码任务、快速验证想法。

踩坑：

- 定制困难：企业内部的私有接口、审批流程、权限校验难以嵌入。
- 黑盒问题：执行失败时难以定位是模型问题、Harness逻辑问题还是工具定义问题。

Agent框架折中方案 - 最终选择

- 路径：使用LangGraph/eino等框架构建Agent
- 特点：平衡了灵活性与开发效率，提供标准化的状态管理、工具调用、循环控制，便于企业集成
- 适用：复杂业务流程、需要精细控制的场景

收获：

- 框架选型看团队技术栈。eino在Go生态表现优异，LangGraph在Python生态更成熟。

总结：平台定位与设计经验

平台定位

企业级管控，而非单纯执行

经验：

- 解决“敢用”与兜底：保障AI操作安全与规范，开源工具假设用户自担风险
- 实现全局可观测：何人，何时，用何技能、做了何事、结果如何

产品设计

从复杂交互回归对话，收敛即效率

经验：

- 最佳交互：当前阶段聊天框，当大模型能力足够强时，自然语言就最好的DSL
- 多交互入口：以机器人的方式嵌入通讯工具中
- 能力越强，交互越收敛：不要复杂的交互界面





总结：架构优化建议

不要为了低质模型过度优化架构，换个模型

踩坑记录：

- 花了大量的时间调整提示词，修格式、补容错
- 改了一版又一版的System Prompt
- 增加了一堆注意事项和禁止行为
- 写了大量重试和修复逻辑

绝大部分换个更强的模型就自然消失了

经验总结：

- 模型是最容易被替换的：今天你为某个模型写的“特调提示词”，明天换了更强的模型可能反而是噪音。
- 工程壁垒在模型之外：工具生态、权限体系、审计日志、可观测性，云端会话存储与管理。

流式输出与事件机制的前端复杂度爆炸

踩坑记录：流式输出（Streaming）和复杂的事件机制，前端不得不承接大量的事件处理逻辑

- 多种事件类型的解析与分发
- 流式消息的拼接与渲染
- 工具调用状态的实时展示
- 异常中断的优雅降级

前端处理代码臃肿，耦合严重，埋下了很深的技术债

经验总结：

- 流式交互场景下，前后端的事件协议必须在架构设计阶段就明确定义，不要让前端成为“事件处理的垃圾桶”。
- 建议抽象统一的事件总线 + 状态机模型，将事件处理逻辑从UI层剥离。

● 整体展望：构建数据驱动的“飞轮”，迈向智能协同

从优化到自进化

让数据反哺知识资产本身：

- 优化记忆、知识抽取：数据导向衡量知识的可用性
- 提高skills迭代效率：skills更新联动，提高迭代效率

从基础到增强

继续推动底层数据完善：

- 业务一致性治理：推动业务向统一架构迁移
- 领域Agent封装：提供封装好的领域Agent



● 整体展望：构建数据驱动的“飞轮”，迈向智能协同

从可用到可度量

建立执行的全链路可观测体系：

- 度量对话质量：任务完成度、主动终断率、事后评分等
- 量化技能效能：耗时指标，调用成功率，识别优劣

从辅助到价值

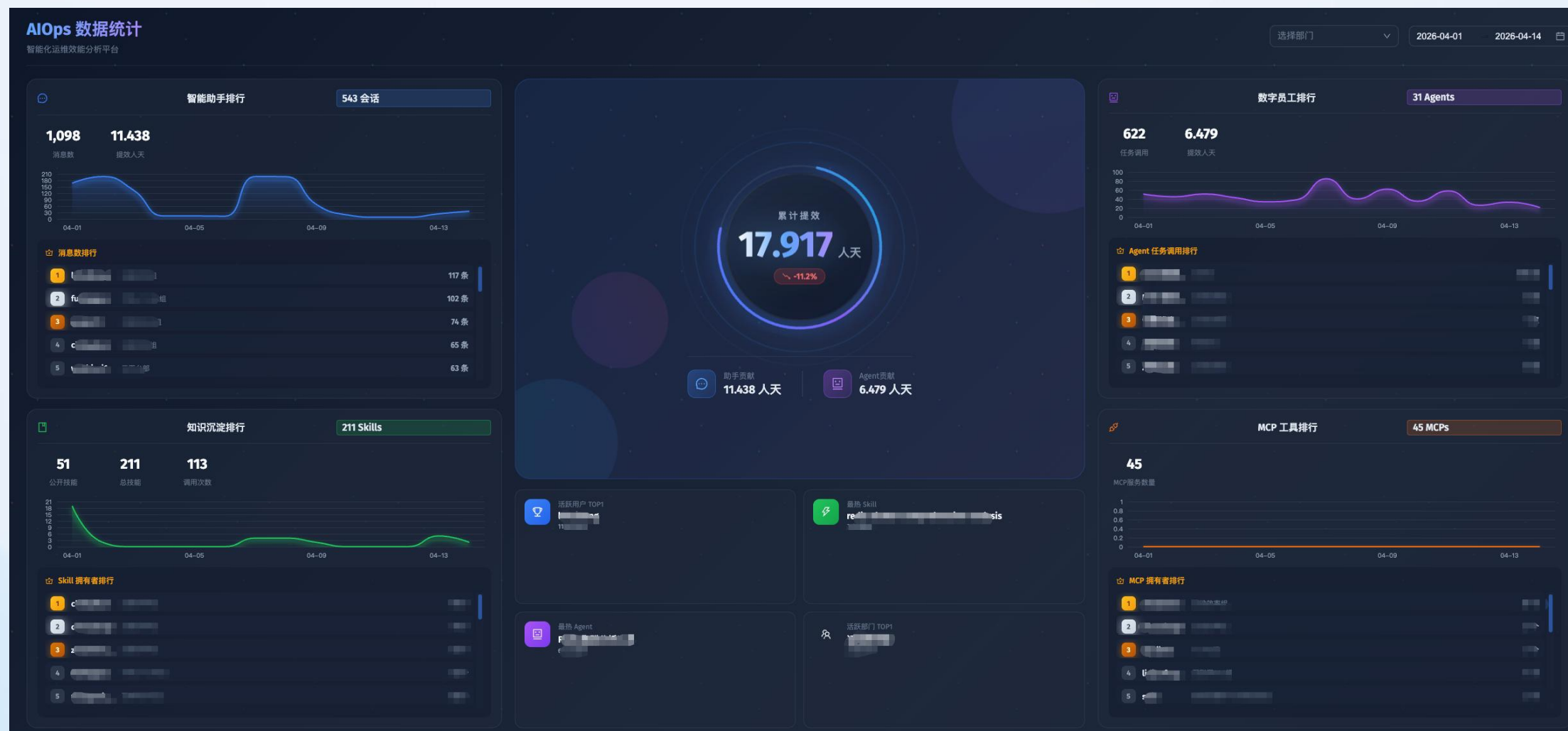
建立人效转化的计算模型：

- 效率价值：AI 执行转化为对于的标准人力
- 质量价值：对比诸如变更失败率等证明质量提升价值





探索测试环境数据大盘 (ing)



THANKS

大模型正在重新定义软件

Large Language Model Is Redefining The
Software



AiCon

全球人工智能开发与应用大会

上海站

探索 AI 应用边界

Explore the limits of AI applications

2026 年 6 月 26-27 日 / 上海张江科学会堂



参会咨询

专题：世界模型与多模态智能突破

专题出品人：朱政

极佳视界 / 联合创始人 & 首席科学家



专题：Agent 系统架构与工程化实践

专题出品人：鲁浩楠

OPPO / 大模型算法负责人



专题：AI 原生数据工程

专题出品人：王彦辉

火山引擎 / 数智平台产品总监



专题：Agent 安全、评测与可信治理

专题出品人：马传雷（岳立）

支付宝 / 业务风险技术部负责人



专题：Agent 数据、记忆与运行时基础设施

专题出品人：邓亚峰

EverMind / CEO



专题：企业级研发体系的重构

专题出品人：沈浪

快手 / 研发效能负责人

